

EXPERIMENT 18

Experiment: Perceptron Based Iris Classification

```
print("THASLIMA NASREEN J/192424152")
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import Perceptron
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
```

Load Iris dataset

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

Convert Iris classification into binary classification

0 = Setosa, 1 = Non-Setosa

```
y_binary = (y != 0).astype(int)
```

Split dataset into training and testing data

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y_binary,
```

```
    test_size=0.2,
```

```
    random_state=42,
```

```
    stratify=y_binary
```

```
)
```

Standardize the features

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```

# Create Perceptron model
model = Perceptron(
    max_iter=1000,
    eta0=0.1,
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Actual Values:")
print(ytest)

print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:", accuracy)

print("Accuracy Percentage:", accuracy * 100, "%")

# Generate confusion matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Display confusion matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Setosa", "Non-Setosa"]
)

disp.plot(cmap="Blues")

plt.title("Perceptron Based Iris Classification")

```

```
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()
```

OUTPUT:

THASLIMA NASREEN J/192424152

Actual Values:

```
[1 1 1 0 1 1 1 0 1 1 0 0 0 1 1 0 1 0 1 0 1 1 0 0 1 1 1 1 1 1]
```

Predicted Values:

```
[1 1 1 0 1 1 1 0 1 1 0 0 0 1 1 0 1 0 1 0 1 1 0 0 1 1 1 1 1 1]
```

Accuracy: 1.0

Accuracy Percentage: 100.0 %

Confusion Matrix:

```
[[10  0]
```

```
[ 0 20]]
```

